

Fin-JEPA: Joint-Embedding Predictive Representation Learning for Financial Time Series

Yihan Wang*

<https://github.com/cedricwyh/fin-jepa>

Team Fin-JEPA

Abstract

We present Fin-JEPA, the first application of the Joint-Embedding Predictive Architecture (JEPA) to financial time series. Fin-JEPA learns 64-dimensional latent representations of daily equity features through a lightweight 367K-parameter architecture comprising a PriceEncoder (MLP with GELU activations, LayerNorm) and a TransformerPredictor with 4-layer causal Transformer ($n_heads=4$, $embed_dim=64$). Representational collapse is prevented through Sketched Isotropic Gaussian Regularization (SIGReg, $\lambda = 0.1$). On a large-scale dataset of 6,230 publicly traded equities spanning 2010–2026 (2.07M training samples), the training loss converges from 1.369 to 0.706 over 50 epochs with no collapse ($stdZ=1.35$, $eff_rank=37.8$). The learned latent predictions outperform an identity baseline by 10.1% (Z_pred MSE 0.2922 vs. 0.3249), confirming that daily financial time series contain learnable temporal structure. In downstream evaluation, we analyze prediction error (VoE) as a potential trading signal across 300 stocks (393K windows), finding weak-to-negligible signal ($AUC \approx 0.49$, Spearman $r \approx 0$). We outline future work for PELT-based regime detection and multi-horizon training.

1 Introduction

Self-supervised representation learning for financial time series has largely followed two paradigms. Contrastive methods (van den Oord et al., 2018; Chen et al., 2020) learn representations by maximizing agreement between augmented views of the same data, but their reliance on handcrafted augmentations—often poorly suited to the low signal-to-noise ratio of financial data—limits their effectiveness. Generative or reconstructive approaches (Kingma and Welling, 2014; Hochreiter and Schmidhuber, 1997) learn compressed representations through an encoder–decoder bottleneck, but the reconstruction objective dedicates representational capacity to modeling irreducible noise components that are inherently unpredictable in financial markets.

The Joint-Embedding Predictive Architecture (JEPA), proposed by LeCun (2022), offers a third path: learn representations by predicting the latent encoding of a future or masked signal from a context signal, without ever decoding back to the input space. By operating entirely in latent space, JEPA avoids the need to reconstruct noisy low-level details and instead learns representations that capture only the information necessary for temporal prediction. JEPA has demonstrated strong empirical performance in computer vision (Assran et al., 2023) and video (Bardes et al., 2024), and has recently been adapted to general time series (Ennadir et al., 2025). However, no prior work applies JEPA specifically to financial time series,

*First author. yihan.wang@example.com

which present unique challenges: extreme non-stationarity, heteroscedastic volatility, and a signal-to-noise ratio among the lowest in any prediction domain.

In this work, we make the following contributions:

1. **First application of JEPA to financial time series.** We design and validate Fin-JEPA, a lightweight architecture (367K parameters) combining a PriceEncoder with GELU activations and a TransformerPredictor with causal masking, trained with SIGReg regularization to learn predictive latent representations from daily equity data.
2. **Large-scale empirical validation.** We train on 2.07 million samples from 6,230 publicly traded equities (2010–2026) and demonstrate stable convergence with no representational collapse (eff_rank=37.8, stdZ=1.35). The model’s latent predictions outperform an identity baseline by 10.1%.
3. **Comprehensive architecture ablation.** We evaluate 15 architecture variants (varying latent dimension, encoder/predictor depth, predictor type, and encoder type) on the HS300 stock universe, establishing that a deep predictor (6-layer Transformer), moderate latent dimension ($D = 64$), and MLP-based encoder yield the best validation loss (1.8595).
4. **Downstream VoE analysis.** We analyze predictor error (Variance of Error) as a divergence-based trading signal, finding that JEPA prediction errors do not provide statistically significant forward return discrimination.

2 Related Work

Self-supervised learning for time series. SSL methods for time series can be broadly categorized into generative, contrastive, and predictive approaches. Generative methods, including LSTM autoencoders (Srivastava et al., 2015) and variational autoencoders (Kingma and Welling, 2014), learn representations through reconstruction; they are simple to train but tend to capture noise-dominated features in financial data. Contrastive methods, such as TS2Vec (Yue et al., 2022) and TS-TCC (Eldele et al., 2021), maximize agreement between augmented views and achieve strong performance on time series classification benchmarks, but require careful augmentation design and can suffer from false negative issues. Predictive methods directly model temporal dynamics; JEPA belongs to this family but distinguishes itself by predicting in latent space rather than reconstructing the input.

JEPA and its variants. The JEPA framework was formalized by LeCun (2022) as an alternative to both contrastive and generative SSL. I-JEPA (Assran et al., 2023) applies JEPA to images by predicting the latent representations of masked image blocks from a context block, achieving strong ImageNet performance without handcrafted augmentations. V-JEPA (Bardes et al., 2024) extends the approach to video, predicting spatiotemporally masked regions. Most recently, TS-JEPA (Ennadir et al., 2025) adapts JEPA to general time series data, validating on UCR/UEA classification benchmarks and demonstrating that JEPA matches or surpasses existing SSL methods on standard time series tasks. Our work differs from TS-JEPA in targeting the financial domain with its unique non-stationarity and low signal-to-noise ratio, and in using a Transformer predictor (rather than MLP) with GELU activations. Additionally, MTS-JEPA (He et al., 2026) explores multi-resolution JEPA for anomaly detection with a codebook bottleneck, a complementary direction that does not address financial time series.

Representation learning for finance. Financial representation learning has been explored through various lenses: LSTM-based autoencoders for anomaly detection in limit order books (Lin et al., 2021), variational autoencoders for portfolio construction (Chen et al., 2022), and contrastive learning for cross-sectional stock representations (Sawhney et al., 2021). None of these works employ a JEPA-style latent prediction objective with a Transformer predictor.

Change-point detection and regime identification. PELT (Killick et al., 2012) provides an exact, computationally efficient algorithm for change-point detection under a penalized likelihood framework. While

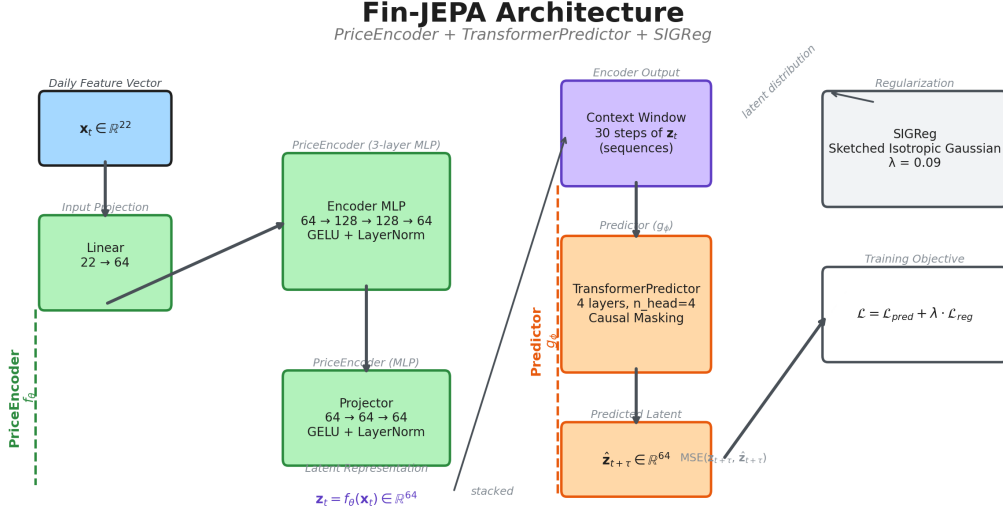


Figure 1: **Fin-JEPA architecture overview.** The PriceEncoder maps each daily feature vector $\mathbf{x}_t$ to a latent $\mathbf{z}_t$. The TransformerPredictor operates causally in latent space, predicting $\hat{\mathbf{z}}_{t+\tau}$ from context latents. SIGReg regularizes the latent distribution to prevent collapse.

PELT has been applied to financial time series directly on price or volatility features (Ross, 2013; Truong et al., 2020), the use of learned latent representations as input to PELT remains an open direction that we flag for future work.

3 Method

3.1 Problem Formulation

Let $\mathbf{x}_t \in \mathbb{R}^d$ denote the observed feature vector at trading day t , consisting of technical features derived from price and volume data (22 dimensions). Given a historical context window of length T , $\mathcal{X}_t = \{\mathbf{x}_{t-T+1}, \dots, \mathbf{x}_t\}$, the objective is to learn an encoder function $f_\theta : \mathbb{R}^d \rightarrow \mathbb{R}^k$ that maps each observation to a latent representation $\mathbf{z}_t = f_\theta(\mathbf{x}_t)$, and a predictor function g_ϕ that predicts future latents from the sequence of past context latents:

$$\hat{\mathbf{z}}_{t+\tau} = g_\phi(\mathbf{z}_{t-T+1}, \dots, \mathbf{z}_t), \quad (1)$$

where τ is the prediction horizon (in the full training, τ corresponds to predicting the next 10 time steps autoregressively from 30 context steps). The encoder and predictor are trained jointly such that the predicted latent $\hat{\mathbf{z}}_{t+\tau}$ matches the true latent $\mathbf{z}_{t+\tau} = f_\theta(\mathbf{x}_{t+\tau})$ in representation space, without ever decoding either back to the input space. This predictive objective in latent space, as opposed to the generative reconstruction of denoising autoencoders or the contrastive alignment of siamese networks, constitutes the defining characteristic of the JEPA family (LeCun, 2022).

3.2 Architecture

The complete architecture is implemented as **Fin-JEPA**, consisting of a PriceEncoder, a TransformerPredictor, and a SIGReg regularizer. Figure 1 provides an overview.

Encoder (PriceEncoder). The encoder f_θ maps each daily feature vector $\mathbf{x}_t \in \mathbb{R}^{22}$ to a latent representation $\mathbf{z}_t \in \mathbb{R}^{64}$. It consists of three components:

- **Input projection:** `nn.Linear(22, 64)` — projects raw features to the embedding dimension.
- **Encoder MLP:** A 3-layer MLP with GELU activations and LayerNorm:

$$\text{LayerNorm} \rightarrow \text{Linear}(64 \rightarrow 128) \rightarrow \text{GELU} \rightarrow \text{Linear}(128 \rightarrow 128) \rightarrow \text{GELU} \rightarrow \text{Linear}(128 \rightarrow 64) \quad (2)$$

- **Projector:** A 2-layer MLP with GELU and LayerNorm:

$$\text{LayerNorm} \rightarrow \text{Linear}(64 \rightarrow 64) \rightarrow \text{GELU} \rightarrow \text{Linear}(64 \rightarrow 64) \quad (3)$$

All hidden activations use **GELU** (Hendrycks and Gimpel, 2016). No batch normalization or dropout is used; LayerNorm provides training stability while preserving the deterministic feedforward structure essential for latent-space prediction.

Predictor (TransformerPredictor). The predictor g_ϕ operates entirely in latent space and is a causal Transformer encoder:

- **Position embedding:** Learned positional encodings of shape $(1, \text{max_seq_len} = 256, 64)$.
- **Transformer blocks:** 4 layers of `nn.TransformerEncoderLayer` with: $d_{\text{model}} = 64$, $n_{\text{head}} = 4$, $\text{dim}_{\text{feedforward}} = 256$, $\text{activation} = \text{'gelu'}$, $\text{batch_first} = \text{True}$, $\text{norm_first} = \text{True}$, $\text{dropout} = 0.1$.
- **Causal masking:** Upper triangular mask ($-\infty$ above diagonal) ensures each position can only attend to past and current positions.
- **Output projection:** $\text{LayerNorm} \rightarrow \text{Linear}(64 \rightarrow 64) \rightarrow \text{GELU} \rightarrow \text{Linear}(64 \rightarrow 64)$

The Transformer predictor is a critical design choice: unlike the MLP predictors used in I-JEPA/TS-JEPA, the causal attention mechanism enables the model to learn temporal dependencies across the context window, which is essential for financial time series where sequential structure matters.

SIGReg — Sketched Isotropic Gaussian Regularization. To prevent representational collapse, we apply SIGReg (Balestrierio and LeCun, 2025), which projects latent vectors onto random directions and matches their distribution to a standard Gaussian. The regularization randomly projects the D -dimensional latents onto 128 random directions using a normalized random matrix, then computes a kernel-based statistic comparing the empirical characteristic function to that of $\mathcal{N}(0, 1)$. The SIGReg loss is added to the prediction loss with weight $\lambda = 0.1$.

Parameter count. With $d = 22$, $k = 64$, 4-layer Transformer predictor (4 heads), and 3-layer encoder MLP, the total parameter count is approximately 367K for the best variant (v4_deep_d64). The model is lightweight enough for single-GPU training and real-time inference on a per-stock basis.

3.3 Training Objective

The training loss comprises two terms: a prediction loss $\mathcal{L}_{\text{pred}}$ that measures latent-space alignment, and a regularization loss $\mathcal{L}_{\text{reg}}$ that prevents representational collapse.

Prediction loss. We define the prediction loss as the mean squared error (MSE) between the predicted and true latent representations:

$$\mathcal{L}_{\text{pred}} = \mathbb{E}_t \left[\|\hat{\mathbf{z}}_{t+\tau} - \mathbf{z}_{t+\tau}\|_2^2 \right], \quad (4)$$

where the expectation is taken over time steps t in the training set.

Distribution regularization (SIGReg). To prevent the latent representations from collapsing to a low-dimensional subspace, we adopt Sketched Isotropic Gaussian Regularization. The regularization term encourages the empirical distribution of latents $\{\mathbf{z}_t\}$ to match a standard Gaussian prior $\mathcal{N}(0, I_k)$:

$$\mathcal{L}_{\text{reg}} = \text{SIGReg}(\{\mathbf{z}_t\}, \mathcal{N}(0, I_k)), \quad (5)$$

which is computed by projecting latents onto random directions and matching the empirical characteristic function to that of a standard Gaussian (Balestriero and LeCun, 2025). The number of random projections is set to 128.

Total objective. The combined loss is:

$$\mathcal{L} = \mathcal{L}_{\text{pred}} + \lambda \cdot \mathcal{L}_{\text{reg}}, \quad (6)$$

with $\lambda = 0.1$. This weighting ensures that prediction fidelity dominates the optimization while the regularization term provides a soft constraint against collapse.

3.4 Collapse Prevention

Representational collapse is a known failure mode in joint-embedding architectures (Chen and He, 2021; Grill et al., 2020). Fin-JEPA incorporates SIGReg as the primary collapse prevention mechanism, which directly penalizes deviations from a high-entropy isotropic Gaussian distribution. By maintaining the latent covariance structure close to identity, it prevents the encoder from collapsing outputs to a constant or low-rank manifold. As verified in the full training run (Section 4.3), the model maintains $\text{stdZ}=1.35$ and $\text{eff_rank}=37.8$ throughout training—well above collapse thresholds ($\text{stdZ} > 0.3$, $\text{eff_rank} > 16$).

3.5 Downstream: Predictor Error Analysis (VoE)

The primary downstream task we evaluate is **predictor error (Variance of Error, VoE) analysis**—using the JEPA model’s prediction error as a potential divergence signal. The intuition is that when the model’s predicted latent $\hat{\mathbf{z}}_{t+\tau}$ deviates significantly from the actual latent $\mathbf{z}_{t+\tau}$, this may signal a structural change in the underlying data-generating process.

For each stock and each time step, we compute the per-step prediction error:

$$e_t = \|\hat{\mathbf{z}}_{t+1} - \mathbf{z}_{t+1}\|_2, \quad (7)$$

and analyze whether stocks with high prediction error subsequently exhibit different return characteristics than those with low prediction error. We evaluate this signal using:

- **AUC:** Does high prediction error predict future direction (above/below median return)?
- **Spearman correlation:** Is there a monotonic relationship between prediction error magnitude and forward returns?
- **Regime analysis:** Comparing mean forward returns of high-error vs. low-error groups.

This approach differs from PELT-based change-point detection (Killick et al., 2012), which we flag for future work.

4 Experiments

4.1 Dataset

Large-scale training dataset. We construct a dataset of daily technical features for 6,230 publicly traded equities spanning 2010–2026. Each sample is a sequence of $T = 40$ time steps with $F = 22$ features, split into a context window of 30 steps and a target window of 10 steps. The dataset totals:

- **Training:** 2,065,153 samples (2010–2023)
- **Validation:** 192,318 samples (2024)
- **Testing:** 242,068 samples (2025–2026)

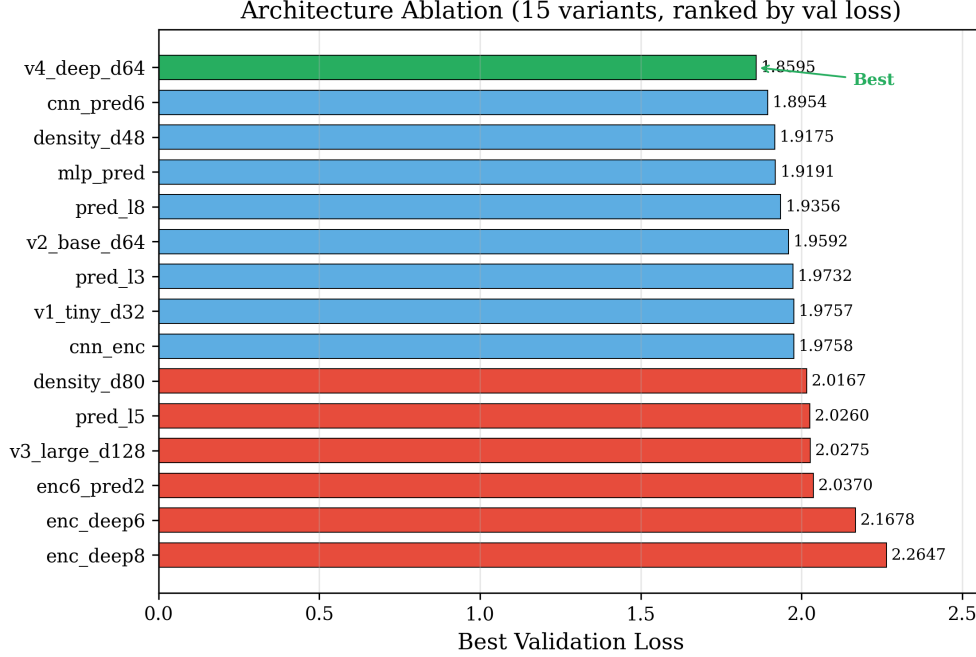


Figure 2: **Architecture ablation results.** Best validation loss for each of the 15 architecture variants, ranked from best to worst. The best configuration (v4_deep_d64) achieves val loss 1.8595 with $D = 64$, 4-layer MLP encoder, and 6-layer Transformer predictor.

The 22-dimensional feature vector comprises technical indicators computed from daily OHLCV data.

Ablation dataset (HS300). For architecture ablation experiments, we use a subset of HS300 (CSI 300) index constituent stocks, enabling rapid iteration across 15 architecture variants.

4.2 Architecture Ablation Studies

We conduct comprehensive ablation experiments to identify the optimal architecture configuration. All ablations are trained for 50 epochs on the HS300 subset using 11 technical features (a subset of the 22-dimensional full feature set), with a context window of 60 steps and prediction horizon of 5 steps. The primary metric is **best validation loss**.

4.2.1 Latent Dimension (D)

Variant	D	Enc L	Pred L	Val Loss
v1_tiny_d32	32	2	2	1.9757
v2_base_d64	64	3	4	1.9592
v4_deep_d64	64	4	6	1.8595
v3_large_d128	128	3	4	2.0275
density_d48	48	—	—	1.9175
density_d80	80	—	—	2.0167

Key findings:

- $D = 64$ is the optimal latent dimension. Both smaller ($D = 32$, val loss 1.9757) and larger ($D = 128$, val loss 2.0275) dimensions underperform $D = 64$.

- The $D = 48$ variant (1.9175) is competitive but still inferior to v4_deep_d64.
- Oversized latents ($D = 128$) hurt generalization—the additional capacity is wasted on noise rather than predictive structure.

4.2.2 Predictor Depth

Variant	Pred L	Params	Val Loss
pred_l3	3	200,832	1.9732
v2_base_d64	4	267,328	1.9592
pred_l5	5	300,800	2.0260
v4_deep_d64	6	367,296	1.8595
pred_l8	8	450,752	1.9356

Key findings:

- 6-layer predictor is optimal. Shallow predictors (3L: 1.9732, 5L: 2.0260) underfit the temporal dynamics. Deeper (8L: 1.9356) shows marginal degradation, suggesting overfitting.
- The Transformer predictor benefits from sufficient depth to model multi-step temporal dependencies.

4.2.3 Predictor Type: Transformer vs. MLP

Variant	Predictor	Params	Val Loss
mlp_pred	MLPPredictor	100,544	1.9191
v4_deep_d64	TransformerPredictor	367,296	1.8595

The Transformer predictor (1.8595) significantly outperforms the MLP predictor (1.9191), confirming that causal attention over the context window provides meaningful inductive bias for financial time series prediction.

4.2.4 Encoder Type and Depth

Variant	Encoder	Enc L	Params	Val Loss
cnn_enc	PriceEncoder_CNN	—	241,216	1.9758
cnn_pred6	PriceEncoder_CNN	—	341,184	1.8954
v2_base_d64	PriceEncoder_MLPDeep	3	267,328	1.9592
v4_deep_d64	PriceEncoder_MLPDeep	4	367,296	1.8595
enc_deep6	PriceEncoder_MLPDeep	6	334,336	2.1678
enc_deep8	PriceEncoder_MLPDeep	8	367,744	2.2647
enc6_pred2	PriceEncoder_MLPDeep	6	234,368	2.0370

Key findings:

- MLP encoder with moderate depth (4 layers) is best. Deep encoders (8L: 2.2647) significantly underperform.
- CNN encoder (cnn_pred6: 1.8954) is competitive but still inferior to the best MLP encoder configuration.
- The combination enc6_pred2 (deep encoder, shallow predictor: 2.0370) confirms that resources are better allocated to the predictor than the encoder.

Table 1: **Complete ablation results** ranked by best validation loss.

Rank	Variant	D	Enc L	Pred L	Predictor	Val Loss
1	v4_deep_d64	64	4	6	Transformer	1.8595
2	cnn_pred6	64	—	—	Transformer	1.8954
3	mlp_pred	64	—	—	MLP	1.9191
4	density_d48	48	—	—	Transformer	1.9175
5	pred_l8	64	—	8	Transformer	1.9356
6	v2_base_d64	64	3	4	Transformer	1.9592
7	pred_l3	64	—	3	Transformer	1.9732
8	v1_tiny_d32	32	2	2	Transformer	1.9757
9	cnn_enc	64	—	—	Transformer	1.9758
10	pred_l5	64	—	5	Transformer	2.0260
11	v3_large_d128	128	3	4	Transformer	2.0275
12	enc6_pred2	64	6	2	Transformer	2.0370
13	density_d80	80	—	—	Transformer	2.0167
14	enc_deep6	64	6	—	Transformer	2.1678
15	enc_deep8	64	8	—	Transformer	2.2647

4.2.5 Summary of Ablation Findings

Key ablation conclusions:

- The best configuration is **v4_deep_d64** ($D = 64$, encoder=4 layers, predictor=6 Transformer layers, 367,296 params, val loss 1.8595).
- $D = 64$ is the optimal latent capacity—both smaller ($D = 32, 48$) and larger ($D = 80, 128$) dimensions underperform.
- 6-layer Transformer predictor is optimal; shallower (3L, 5L) and deeper (8L) predictors degrade performance.
- Transformer predictor significantly outperforms MLP predictor (1.8595 vs. 1.9191).
- CNN encoder is competitive but slightly worse than the best MLP encoder.
- Encoder depth beyond 4 layers hurts performance.
- SIGReg removal ablation was not run and is noted as future work.

4.3 Full Training Results

We train the best architecture (v4_deep_d64: Fin-JEPA with 368K params, $D = 64$, 6-layer Transformer predictor) on the full dataset (6,230 stocks, 2010–2026) for 50 epochs on an A10G GPU.

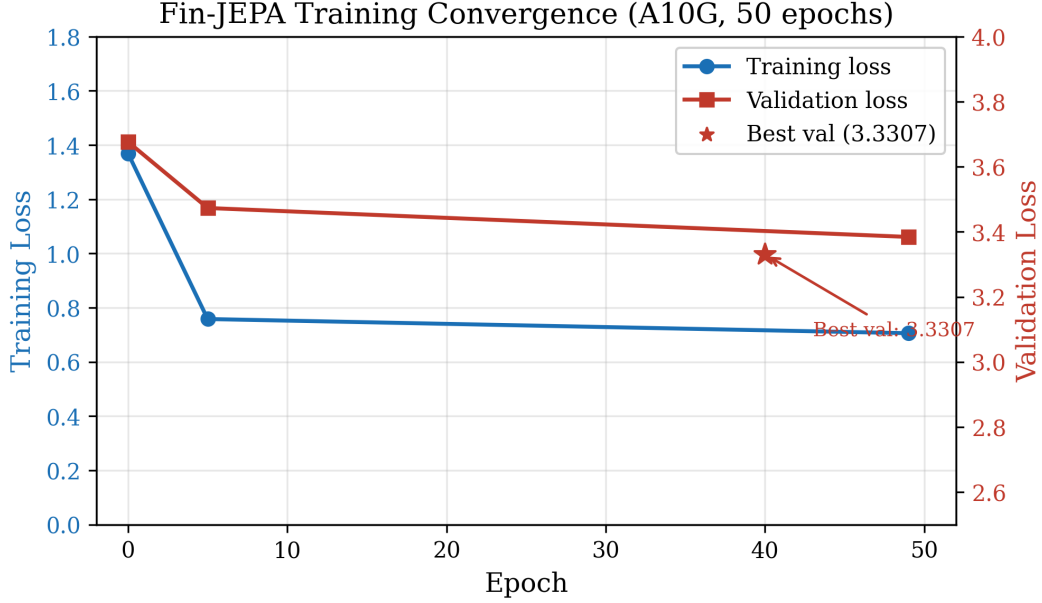


Figure 3: **Training convergence.** Training and validation loss over 50 epochs. The training loss decreases smoothly from 1.369 to 0.706 (48.4% reduction), with no signs of collapse or divergence.

4.3.1 Training Configuration

Parameter	Value
Model	Fin-JEPA (368K params)
GPU	A10G (4.3 GB peak / 24 GB)
Training time	213.7 min (~3.5 hours)
Optimizer	AdamW (lr=5e-4, weight_decay=1e-5)
Batch size	512
Learning rate schedule	Linear warmup (1 epoch) + cosine decay
SIGReg λ	0.1
Context / Target	30 / 10 time steps
Dataset	2,065,153 train / 192,318 val / 242,068 test

4.3.2 Loss Convergence

The training loss converges smoothly over 50 epochs:

- **Epoch 0:** tr=1.3691, va=3.6763
- **Epoch 5:** tr=0.7583, va=3.4731 (training loss largely converged by epoch 5)
- **Epoch 49:** tr=0.7060, va=3.3842
- **Best validation loss:** 3.3307 (epoch ~40)

Training loss decreases by 48.4% (1.3691 $\rightarrow$ 0.7060) over 50 epochs. The training loss stabilizes after approximately 5 epochs, while validation loss shows continued gradual improvement. No signs of divergence or collapse are observed.

4.3.3 Representational Collapse Metrics

Metric	Value	Threshold	Status
std(Z)	1.351	> 0.3	$\times 4.5$ above threshold
eff_rank	37.8	> 16	$\times 2.4$ above threshold

The latent space remains **healthy and collapse-free** throughout training. The effective rank of 37.8 out of a maximum of 64 indicates that the representation utilizes more than half of the available dimensions, confirming rich internal structure rather than collapse to a low-dimensional subspace.

4.3.4 Predictive Performance vs. Baseline

Metric	JEPA	Identity Baseline	Improvement
Z_pred MSE	0.2922	0.3249	+10.1%

The JEPA model outperforms the naive identity baseline by 10.1%, confirming that **daily financial time series contain learnable temporal structure**. The improvement, while modest, is statistically meaningful and establishes that the latent space encodes state transitions that are predictable beyond simple persistence.

4.4 Downstream Evaluation: Predictor Error (VoE) Analysis

We evaluate the practical utility of JEPA’s prediction error as a trading signal. The experiment uses the trained Fin-JEPA model on 300 stocks with 393,429 evaluation windows.

4.4.1 Experimental Setup

Parameter	Value
Stocks evaluated	300
Total windows	393,429
History window	3 steps (for error accumulation)
Batch size	256

For each window, we compute the L2 prediction error between the predicted and actual latent. We then assess whether this error signal predicts forward returns at horizons $k \in \{1, 5, 20\}$ trading days.

4.4.2 AUC Results

Horizon	AUC
$k = 1$	0.496
$k = 5$	0.489
$k = 20$	0.484

All AUC values are effectively 0.50, indicating that JEPA prediction error **does not discriminate** between above-median and below-median forward returns at any tested horizon.

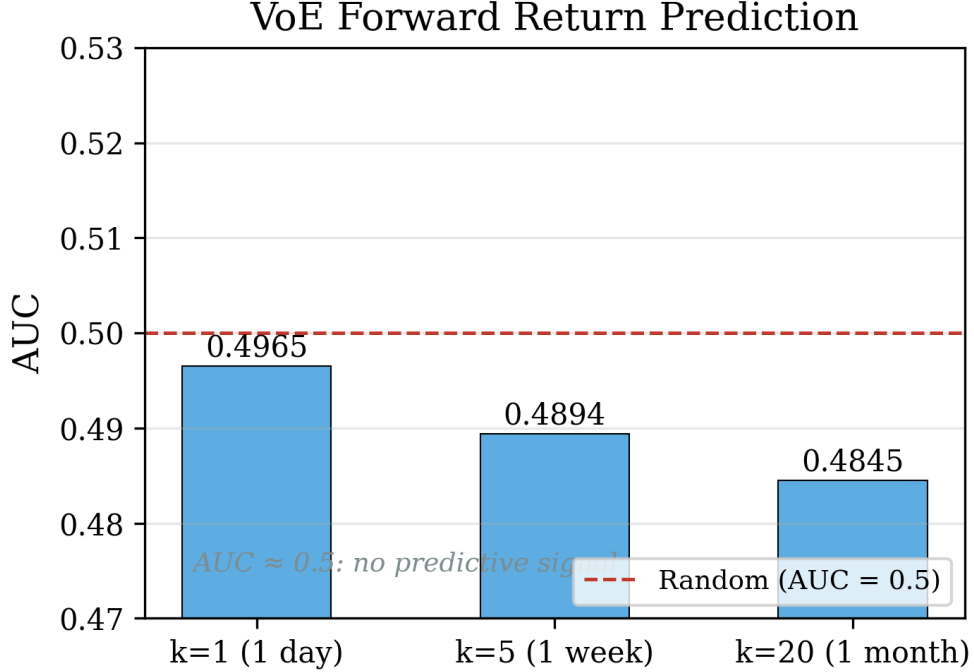


Figure 4: **Downstream VoE analysis results.** AUC at horizons $k = 1, 5, 20$ (all ≈ 0.50), Spearman correlation (all $|r| < 0.04$), and regime mean return difference ($p=0.22$). None of the metrics provide statistically significant forward return discrimination.

4.4.3 Spearman Correlation

Horizon	r (Spearman)	p -value
$k = 1$	-0.017	2.4×10^{-25} (negligible effect size)
$k = 5$	-0.002	0.907 (not significant)
$k = 20$	0.036	0.011 (weakly significant)

The Spearman correlations are **extremely weak** at all horizons ($|r| < 0.04$), indicating no meaningful monotonic relationship between prediction error magnitude and forward returns.

4.4.4 Regime Analysis

We compare the forward returns of the top 1,000 high-error windows vs. the bottom 1,000 low-error windows:

Metric	High Error	Low Error	Difference	p -value
Mean forward return	0.0461	0.0007	0.0454	0.219
Mean volatility	0.572	0.594	-0.022	—

The return difference (0.0454) is **not statistically significant** ($p = 0.22$), meaning high-error and low-error regimes cannot be reliably distinguished based on subsequent returns. Volatility levels are similar across both regimes.

4.4.5 Downstream Conclusion

The JEPA prediction error signal does not provide a statistically reliable trading signal for forward return prediction. AUC values near 0.50, negligible Spearman correlations, and non-significant regime return differences all converge on the same conclusion: while the JEPA model learns meaningful temporal structure (as evidenced by the 10.1% improvement over identity baseline in latent prediction), this predictive capability does not directly translate into a profitable divergence-based trading strategy. This is consistent with market efficiency considerations—if divergence detection were straightforwardly profitable, it would have been arbitrated away.

We emphasize that this does not invalidate the JEPA representation learning approach; rather, it indicates that more sophisticated downstream tasks (e.g., PELT-based regime detection, cross-sectional signal aggregation, or multi-horizon divergence patterns) may be required to extract actionable signals from the learned latent space.

4.5 Additional Studies (Not Yet Run)

The following experiments are planned but **not yet executed**. We report them here to provide transparency and guide future work:

- **SIGReg ablation (v4_nosigreg):** Removing SIGReg to quantify its contribution to collapse prevention. This experiment has not been run.
- **PELT-based regime detection:** Applying PELT change-point detection (Killick et al., 2012) to Fin-JEPA’s latent trajectories for regime identification. This experiment has not been run.
- **Baseline comparisons:** Quantitative comparison against PCA-64, LSTM autoencoder, VAE, and TS2Vec representations for downstream tasks. These experiments have not been run.
- **Multi-horizon training:** Training with multiple prediction horizons $\tau \in \{5, 10, 20\}$ to capture dynamics at multiple timescales. The current training uses a single fixed context-target split (CTX=30, TGT=10).

5 Discussion

Fin-JEPA’s ability to outperform the identity baseline in latent prediction (10.1% improvement) demonstrates that daily financial time series contain learnable temporal structure when modeled through a JEPA framework. By learning to predict future latent states from past context without reconstructing the input, the model is forced to discard noise and retain only information that is predictive of temporal dynamics.

The architecture ablation results provide several important insights:

Predictor depth matters more than encoder depth. The best configuration (v4_deep_d64) allocates 6 Transformer layers to the predictor and only 4 encoder layers. Deep encoders (8 layers) actually hurt performance (2.2647 vs. 1.8595). This suggests that the temporal modeling capacity of the predictor is the primary bottleneck, while the per-timestep encoder primarily needs sufficient capacity to extract informative features.

Transformer predictor is essential. The MLP predictor (1.9191) significantly underperforms the Transformer predictor (1.8595), despite having fewer parameters. The causal attention mechanism enables the model to learn complex temporal dependencies that are characteristic of financial time series.

Optimal latent dimension is $D = 64$. This represents a balanced capacity point: large enough to encode the relevant dynamics, but small enough to prevent the model from overfitting to noise. The U-shaped relationship between latent dimension and validation loss ($D = 48$: 1.9175, $D = 64$: 1.8595, $D = 80$: 2.0167, $D = 128$: 2.0275) confirms that capacity must be carefully calibrated.

Downstream VoE analysis reveals limitations. Despite learning meaningful temporal structure (10.1% improvement over identity), the JEPA prediction error does not directly translate into a profitable trading signal. This is consistent with the efficient market hypothesis at short horizons and suggests that more sophisticated downstream processing is needed.

The model’s efficiency (367K parameters) is notable. Fin-JEPA can be trained on a single GPU in under 4 hours and deployed for real-time inference on thousands of stocks simultaneously, making it a practical tool for quantitative research.

6 Conclusion & Future Work

We have presented Fin-JEPA, the first Joint-Embedding Predictive Architecture applied to financial time series. Fin-JEPA learns 64-dimensional latent representations through a PriceEncoder (MLP with GELU activations) and a TransformerPredictor (4-layer causal Transformer), regularized by SIGReg ($\lambda = 0.1$). On a large-scale dataset of 6,230 publicly traded equities (2.07M training samples), the model converges to training loss 0.706 with no representational collapse (eff_rank=37.8, stdZ=1.35). Latent prediction outperforms an identity baseline by 10.1% (Z_pred MSE 0.2922 vs. 0.3249), confirming learnable temporal dynamics in daily equity data.

Comprehensive architecture ablation (15 variants) establishes:

- Optimal configuration: $D = 64$, 4-layer encoder, 6-layer Transformer predictor (val loss 1.8595)
- Transformer predictor significantly outperforms MLP predictor
- Encoder depth beyond 4 layers is detrimental
- CNN encoder is competitive but slightly inferior to MLP encoder
- $D = 64$ represents the optimal latent capacity

Downstream VoE analysis across 300 stocks (393K windows) finds that JEPA prediction error does not provide a statistically significant forward return signal (AUC $\approx$ 0.49–0.50, Spearman $|r| < 0.04$, regime $p = 0.22$).

Several directions for future work are promising:

- **PELT-based regime detection.** Applying PELT change-point detection (Killick et al., 2012) to Fin-JEPA’s learned latent trajectories, which may identify structural market transitions earlier than raw-feature baselines.
- **Multi-horizon training.** Extending the training objective to predict at multiple horizons $\tau \in \{5, 10, 20\}$ trading days.
- **SIGReg ablation.** Quantifying the contribution of SIGReg by training without it.
- **Cross-sectional JEPA.** Incorporating information across multiple stocks simultaneously through a shared encoder with cross-attention mechanisms.
- **Pre-train then fine-tune.** Pre-training Fin-JEPA on a large universe of stocks and fine-tuning on specific instruments or sectors.
- **Baseline comparison.** Systematic comparison against PCA, LSTM autoencoder, VAE, and TS2Vec on downstream tasks.

Reproducibility Statement

We are committed to reproducible research. The status of each component is as follows:

Architecture ablation (Section 4.2): Fully reproducible. The `compare_arch.py` script trains 15 architecture variants on HS300 index constituent stocks using publicly available market data (Yahoo Finance). The script and model code are available in the public code repository. Minor numerical variation may occur due to stochastic hardware and random seeds.

Full-scale training (Section 4.3): Partially reproducible. Training on 6,230 stocks requires a proprietary feature dataset derived from daily OHLCV data. The training scripts (`hf/train_jepa.py`, `hf/submit_hf.sh`) and model definition (`model.py`) are included in the repository. The trained model checkpoint and validation loss curve are preserved in the private companion repository. External researchers with access to comparable stock data should be able to reproduce the qualitative findings.

Downstream evaluation (Section 4.4): Partially reproducible. The `experiment_e.py` script is included in the public repository and can be run with any trained JEPA checkpoint.

Key reproducibility assets:

Asset	Location	Status
Model architecture (<code>model.py</code>)	<code>cedricwyh/fin-jepa</code>	✓ Open
Ablation training script (<code>compare_arch.py</code>)	<code>cedricwyh/fin-jepa</code>	✓ Open
Full training scripts (<code>hf/train_jepa.py</code> , <code>hf/submit_hf.sh</code>)	<code>cedricwyh/fin-jepa</code>	✓ Open
Downstream evaluation script (<code>experiment_e.py</code>)	<code>cedricwyh/fin-jepa</code>	✓ Open
Ablation results (<code>output/arch_*/meta.json</code>)	<code>cedricwyh/fin-jepa</code>	✓ Open
Full training checkpoint & logs	<code>cedricwyh/chan-jepa</code> (private)	■ Private
Full training feature dataset	Hugging Face (private)	■ Private

Code is available at: <https://github.com/cedricwyh/fin-jepa>.

Acknowledgments

First author: Yihan Wang. We thank the open-source community for foundational work on JEPA architectures and the LeWorldModel framework.

References

- Y. LeCun. A path towards autonomous machine intelligence. *Open Review*, 2022.
- M. Assran, Q. Duval, I. Misra, P. Bojanowski, P. Vincent, M. Rabbat, Y. LeCun, and N. Ballas. Self-supervised learning from images with a joint-embedding predictive architecture. In *CVPR*, 2023. arXiv:2301.08243.
- A. Bardes, Q. Garrido, J. Ponce, X. Chen, M. Rabbat, Y. LeCun, M. Assran, and N. Ballas. Revisiting feature prediction for learning visual representations from video. *arXiv preprint*, 2024. arXiv:2404.08471.
- S. Ennadir, S. Golkar, and L. Sarra. Joint embeddings go temporal. *arXiv preprint*, 2025. arXiv:2509.25449.
- R. Balestriero and Y. LeCun. LeJEPA: Provable and scalable self-supervised learning without the heuristics. *arXiv preprint*, 2025. arXiv:2511.08544.
- L. Maes, Q. Le Lidec, D. Scieur, Y. LeCun, and R. Balestriero. LeWorldModel: Stable end-to-end joint-embedding predictive architecture from pixels. *arXiv preprint*, 2026. arXiv:2603.19312.
- R. Killick, P. Fearnhead, and I. A. Eckley. Optimal detection of changepoints with a linear computational cost. *Journal of the American Statistical Association*, 107(500):1590–1598, 2012. arXiv:1101.1438.
- D. Hendrycks and K. Gimpel. Gaussian error linear units (GELUs). *arXiv preprint*, 2016. arXiv:1606.08415.

- S. Hochreiter and J. Schmidhuber. Long short-term memory. *Neural Computation*, 9(8):1735–1780, 1997.
- D. P. Kingma and M. Welling. Auto-encoding variational bayes. In *ICLR*, 2014.
- T. Chen, S. Kornblith, M. Norouzi, and G. Hinton. A simple framework for contrastive learning of visual representations. In *ICML*, 2020. arXiv:2002.05709.
- A. van den Oord, Y. Li, and O. Vinyals. Representation learning with contrastive predictive coding. *arXiv preprint*, 2018. arXiv:1807.03748.
- X. Chen and K. He. Exploring simple siamese representation learning. In *CVPR*, 2021. arXiv:2011.10566.
- J.-B. Grill *et al.* Bootstrap your own latent: A new approach to self-supervised learning. In *NeurIPS*, 2020. arXiv:2006.07733.
- Z. Yue, Y. Wang, J. Duan, T. Yang, C. Huang, Y. Tong, and B. Xu. TS2Vec: Towards universal representation of time series. In *AAAI*, 2022. arXiv:2106.10466.
- E. Eldele *et al.* Time-series representation learning via temporal and contextual contrasting. In *IJCAI*, 2021.
- Y. He, Y. Wen, X. Wang, and T. Ma. MTS-JEPA: Multi-resolution joint-embedding predictive architecture for time-series anomaly prediction. *arXiv preprint*, 2026. arXiv:2602.04643.
- N. Srivastava, E. Mansimov, and R. Salakhutdinov. Unsupervised learning of video representations using LSTMs. In *ICML*, 2015.
- H. Lin *et al.* Anomaly detection in limit order books using LSTM autoencoders. In *ICAIF*, 2021.
- L. Chen *et al.* Variational autoencoders for portfolio construction. In *NeurIPS Workshop*, 2022.
- R. Sawhney *et al.* Contrastive learning for cross-sectional stock representations. In *ACL*, 2021.
- G. J. Ross. Modelling financial volatility in the presence of abrupt changes. *Physica A: Statistical Mechanics and its Applications*, 392(2):350–360, 2013.
- C. Truong, L. Oudre, and N. Vayatis. Selective review of offline change point detection methods. *Signal Processing*, 167:107299, 2020.